

ATS Integration Flow Document

Day 16 – Backend Integration Planning

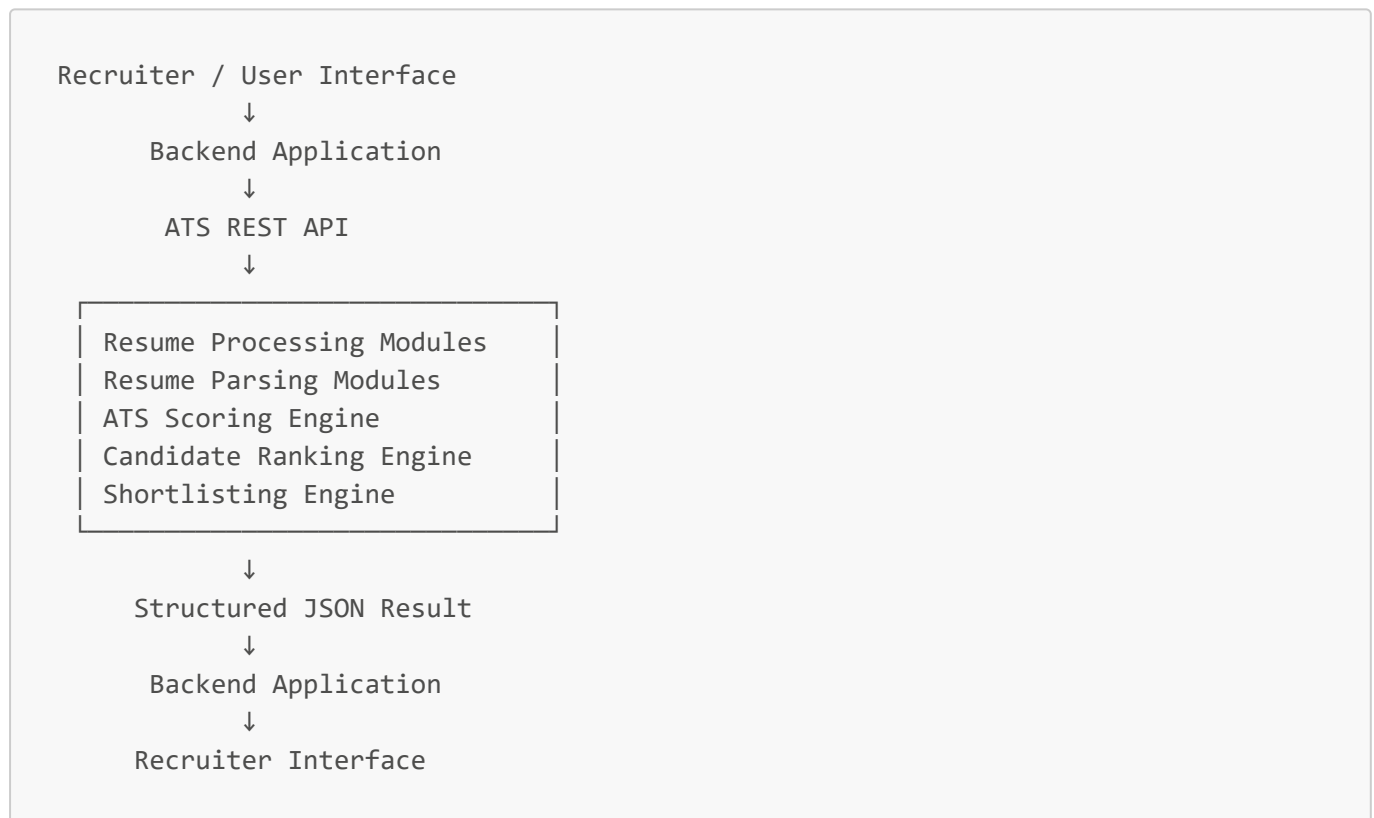
1. Overview

The ATS integration flow defines how a backend application communicates with the resume screening modules through the REST API.

The API layer provides a controlled interface between external application components and the existing AI-based resume evaluation pipeline.

The design supports resume upload, parsing, ATS scoring, candidate ranking and shortlisting, asynchronous processing, structured errors, and application logging.

2. High-Level Integration Architecture



The backend does not need to directly call individual AI modules.

Instead, it communicates through the ATS REST API.

3. End-to-End Candidate Flow

The planned candidate evaluation workflow is:

```
Resume Upload
  ↓
Resume Processing Job
  ↓
Resume Parsing
  ↓
Structured Candidate Information
  ↓
Candidate Evaluation
  ↓
ATS Scoring
  ↓
Candidate Ranking
  ↓
Shortlisting
  ↓
Recruiter-Friendly Output
```

4. Step 1 – Resume Upload

The backend sends the candidate resume to:

```
POST /api/v1/resumes/upload
```

The API validates:

- File presence
- Supported file extension
- Non-empty file content

Supported formats are:

```
PDF
DOCX
```

If accepted, the API generates:

```
resume_id
job_id
```

and initially returns:

QUEUED

5. Step 2 – Asynchronous Processing

Resume processing can involve file extraction, parsing, normalization, and other operations.

The Day 16 API therefore demonstrates asynchronous processing using background tasks.

Job lifecycle:

```
graph TD;
  QUEUED --> PROCESSING;
  PROCESSING --> COMPLETED;
```

Failure path:

```
graph TD;
  PROCESSING --> FAILED;
```

The backend can retrieve the status through:

```
GET /api/v1/jobs/{job_id}
```

6. Step 3 – Resume Parsing

Resume text can be submitted to:

```
POST /api/v1/resumes/parse
```

The endpoint defines the contract for integration with the existing resume parser pipeline.

Input:

```
{  
  "resume_text": "Candidate resume content"  
}
```

The parsing layer can later connect the API with existing resume extraction and parsing components.

7. Step 4 – Candidate Evaluation

Parsed resume information can be evaluated using existing screening components.

Relevant evaluation dimensions include:

```
Skill Match  
Experience Relevance  
Education Alignment  
Semantic Similarity
```

These evaluation results become inputs to the ATS Scoring Engine.

8. Step 5 – ATS Scoring

The backend submits candidate evaluation scores to:

```
POST /api/v1/scoring
```

Example:

```
{  
  "job_role": "Data Scientist",  
  "scores": {  
    "skill_match": 90,  
    "experience_relevance": 80,  
    "education_alignment": 75,  
    "semantic_similarity": 85  
  }  
}
```

The API passes the individual score values to the existing ATS Scoring Engine.

The tested result was:

Overall ATS Score: 85.0
Recommendation: Strong Candidate

9. Step 6 – Candidate Ranking and Shortlisting

After multiple candidates receive ATS scores, candidate information can be submitted to:

POST /api/v1/shortlisting

The shortlisting module:

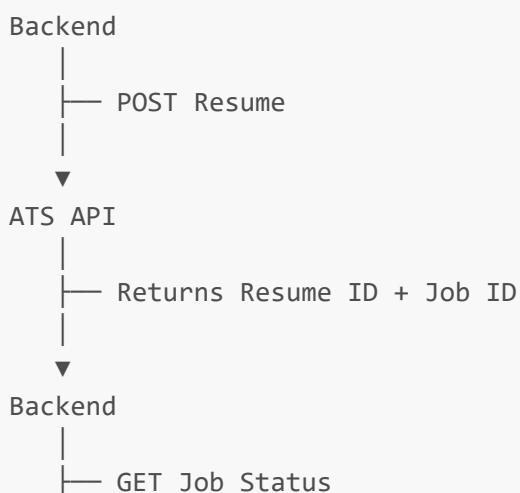
1. Receives candidate records.
2. Uses ATS scores for ranking.
3. Sorts candidates from highest to lowest score.
4. Applies configured thresholds.
5. Assigns candidate decisions.
6. Generates a top-candidate list.

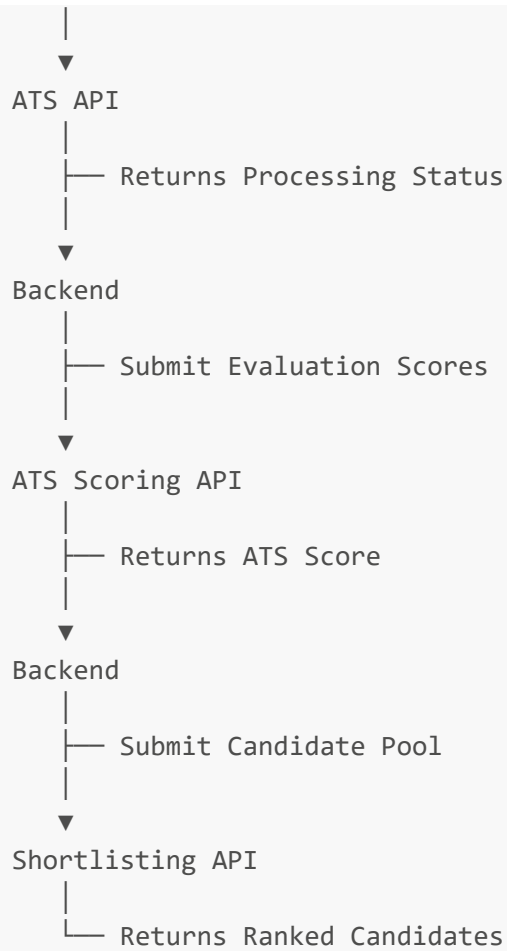
Possible decisions are:

Shortlisted
Review
Rejected

10. Backend Interaction Flow

A typical backend interaction can be represented as:

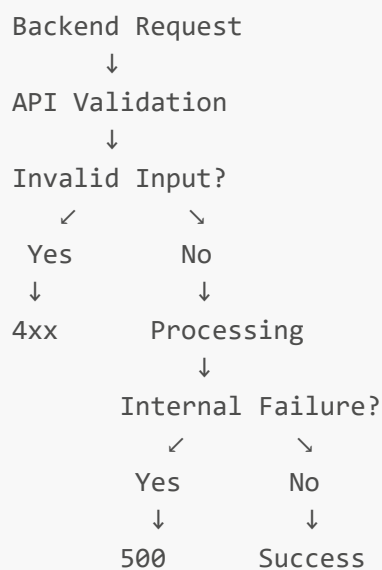




11. Error Flow

The integration design also defines predictable failure behavior.

Example:



Typical responses include:

```
400 - Invalid request
404 - Resource not found
422 - Validation error
500 - Internal processing error
```

12. Logging Flow

Important API operations are recorded using the existing logging system.

Examples include:

```
API request received
Resume upload accepted
Processing job created
Processing started
Processing completed
ATS scoring completed
Candidate shortlisting completed
Processing failed
Resource not found
```

The API should avoid logging:

- Full raw resume content
- Sensitive personal information
- Credentials
- Authentication tokens

This helps maintain safer operational logging practices.

13. API Versioning

The API uses:

```
/api/v1
```

Versioning allows future API changes without immediately breaking existing backend integrations.

Future versions could use:

```
/api/v2
```

while maintaining compatibility with existing clients where necessary.

14. Current Day 16 Scope

The current implementation provides an integration-ready API design and demonstrates connectivity with existing scoring and shortlisting modules.

The resume parsing endpoint currently represents an integration point for the existing parser pipeline rather than replacing those modules.

The asynchronous implementation uses FastAPI background tasks and temporary in-memory storage for development purposes.

15. Production Integration Considerations

A production deployment could later introduce:

- Persistent database storage
- Object/file storage for resumes
- Authentication and authorization
- Dedicated background workers
- Message queues
- Job retry mechanisms
- Rate limiting
- Request tracing
- Centralized monitoring
- API gateway integration
- Secure file validation
- Persistent job history

These are future deployment considerations and are not required for the current Day 16 implementation.

16. Testing and Validation

The Day 16 integration was tested across the primary API behaviors.

Successful tests included:

```
Health endpoint  
Resume parsing endpoint
```



```
ATS scoring endpoint  
Candidate shortlisting endpoint  
Request validation  
Job-not-found handling
```

Final test result:

```
All ATS API Design and Integration tests passed successfully!
```

The ATS scoring integration returned:

```
Job Role: Data Scientist  
Overall Score: 85.0  
Recommendation: Strong Candidate
```

This confirms that the API successfully communicates with the existing ATS Scoring Engine.

17. Conclusion

The Day 16 integration design establishes a REST-based communication layer between backend systems and the ATS resume screening modules.

The architecture provides structured endpoints, request and response contracts, asynchronous job handling, validation, error standards, and logging guidelines.

This design creates a foundation for connecting the existing AI evaluation modules with future backend services and recruiter-facing application interfaces without requiring those external systems to directly interact with individual internal modules.